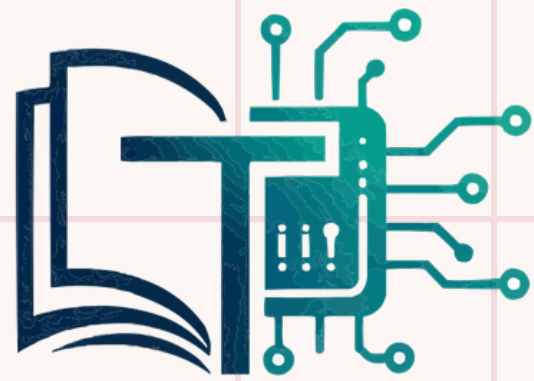




Tech & Teach

T² PROCESSOR

A 4-bit RTL Processor — Design, Verification & Implementation

Group Name: Logic ladies

INTRODUCTION

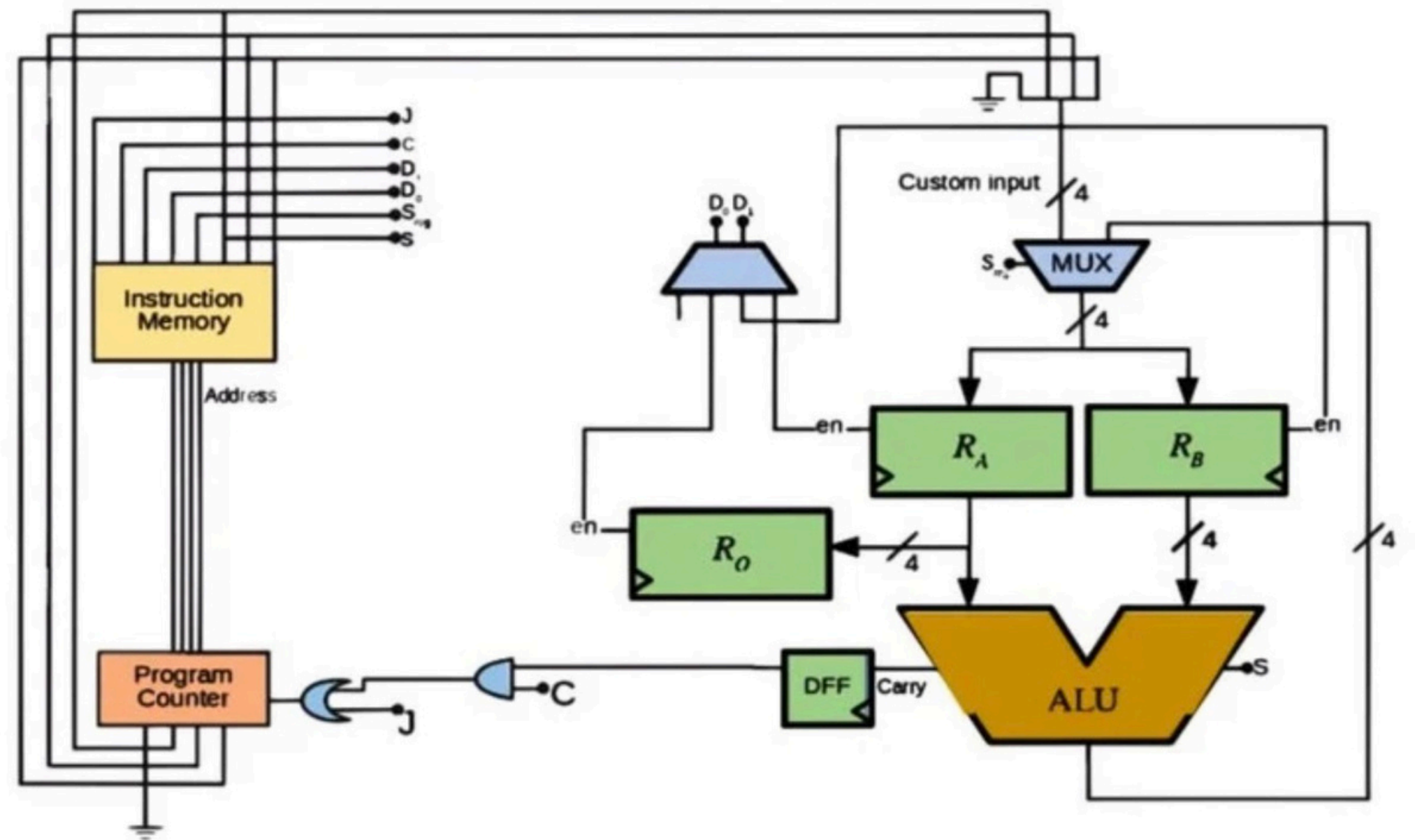
This project focuses on designing and implementing a T2 processor using SystemVerilog. The main goal is to build a functional processor capable of executing instructions, performing arithmetic operations, and handling control flow. The processor was tested through simulation using a Fibonacci sequence.

DATAPATH DESIGN PHASE 1



COMPONENT (LOGIC CIRCUIT) USED PHASE-1

1. INSTRUCTION MEMORY
2. COUNTER (PC)
3. DECODER
4. MULTIPLEXER (MUX)
5. REGISTER R_0
6. REGISTER R_a
7. REGISTER R_β
8. ARITHMETIC LOGIC UNIT (ALU)
9. REGISTER (DFF)
10. CONTROLER



ALU (alu.sv)

```
module Alu #(parameter WIDTH=4)(
input logic [WIDTH-1:0] A, // from Ra
input logic [WIDTH-1:0] B, // from Rb
input logic S,
output logic [WIDTH-1:0] F,
output logic carry
);

logic [3:0] b_final;

assign b_final = B ^ {4{S}};

assign {carry, F} = A + b_final + S;

endmodule
```

Counter (counter.sv)

```
module program_counter #(
parameter WIDTH = 4)(
input logic clk,
input logic rstn,
input logic en,
input logic [WIDTH-1:0] D,
output logic [WIDTH-1:0] Q
);
always_ff @(posedge clk or negedge rstn) begin
if (!rstn)
Q <= '0;
else if (en)
Q <= D;
else
Q <= Q + 1'b1;
end
endmodule
```

Multiplexer (mux.sv)

```
module mux #(
parameter WIDTH = 4)(
input logic [WIDTH-1:0] W0,
input logic [WIDTH-1:0] W1,
input logic Sel,
output logic [WIDTH-1:0] F
);
assign F = Sel? W1 : W0;
endmodule
```

Registers (registers.sv)

```
module file_registers #(
parameter WIDTH = 4
)(
input logic clk,
input logic rstn,
input logic en,
input logic [WIDTH-1:0] data_in,
output logic [WIDTH-1:0] data_out
);
always_ff @(posedge clk or negedge rstn)
begin
if (!rstn)
data_out <= '0;
else if (en)
data_out <= data_in;
end
endmodule
```

InstructionMemory (instruction_memory.sv)

```
module instruction_memory #(
parameter IMEM_DEPTH = 16,
parameter FILE_NAME =
"C:\\Users\\alfra\\Downloads\\machine_code.mem"
)(
input logic [$clog2(IMEM_DEPTH)-1:0] address,
output logic [7:0] instruction
);

logic [7:0] imem [0:IMEM_DEPTH-1];
assign instruction = imem[address];
initial
$readmemb(FILE_NAME, imem);
endmodule
```

Decoder (Decoder.sv)

```
module Decoder (
input logic D1, D0,
output logic y0,y1,y2,y3
);
always_comb begin
y0=(~D1 & ~D0);
y1=(~D1 & D0);
y2=(D1 & ~D0);
y3= (D1 & D0);
end
endmodule
```

Control Unit (control_unit.sv)

```
module control_unit (
input logic [7:0] instruction,
output logic J,
output logic C,
output logic D1,
output logic D0,
output logic Sreg,
output logic S,
output logic [2:0] imm
);

always_comb begin
J = instruction[7];
C = instruction[6];
D1 = instruction[5];
D0 = instruction[4];
Sreg = instruction[3];
S = instruction[2];
imm = instruction[2:0];
end
endmodule
```

PC (PC.sv)

```
module PC #(
parameter WIDTH = 4)(
input logic clk,
input logic rstn,
input logic load,
input logic [WIDTH-1:0] D,
output logic [WIDTH-1:0] Q);
always_ff @(posedge clk or negedge rstn)
begin
if (!rstn)
Q <= 0;
else if (load)
Q <= D;
else
Q <= Q + 1'b1;
end
endmodule
```


machine code

Fibonacci Numbers (Phase 1 testing)

00001000

00011001

00100000

00010000

01110000

00000000

00010100

00000100

10110010

- 0 RA = 0 00001000 load zero to A
- 1 RB = 1 00011001 load 1 to B
- 2 RO = RA 00100000 push A to output
- 3 RB = RA + RB 00010000 Add A to B
- 4 jump if carry 01110000 If A+B produce carry jump to start
- 5 RA = RA + RB 00000000 Swap A and B
- 6 RB = RA — RB 00010100 Swap A and B
- 7 RA = RA — RB 00000100 Swap A and B
- 8 jump to 2 10110010 jump to 3rd instruction

Top (topT.sv)

```
module topT #(
parameter WIDTH = 4,
parameter IMEM_DEPTH = 16,
parameter FILE_NAME =
"C:\\Users\\alfra\\Downloads\\machine_code.mem")(
input logic clk,
input logic rstn,
output logic [WIDTH-1:0] RO,
output logic Carry);
// Internal signals
logic [7:0] instruction;
logic J;
logic C;
logic D1;
logic D0;
logic Sreg;
logic S;
logic [2:0] imm;
//Datapath
datapath #(
.WIDTH(WIDTH),
.IMEM_DEPTH(IMEM_DEPTH),
.FILE_NAME(FILE_NAME))
DP (
.clk(clk),
.rstn(rstn),
.J(J),
.C(C),
.D1(D1),
.D0(D0),
.Sreg(Sreg),
.S(S),
.instruction(instruction),
.RO(RO),
.Carry(Carry),
.imm(imm));
// Control Unit
control_unit CU (
.instruction(instruction),
.J(J),
.C(C),
.D1(D1),
.D0(D0),
.Sreg(Sreg),
.S(S),
.imm(imm) );
endmodule
```

Top Testbenche(topT_tb.sv)

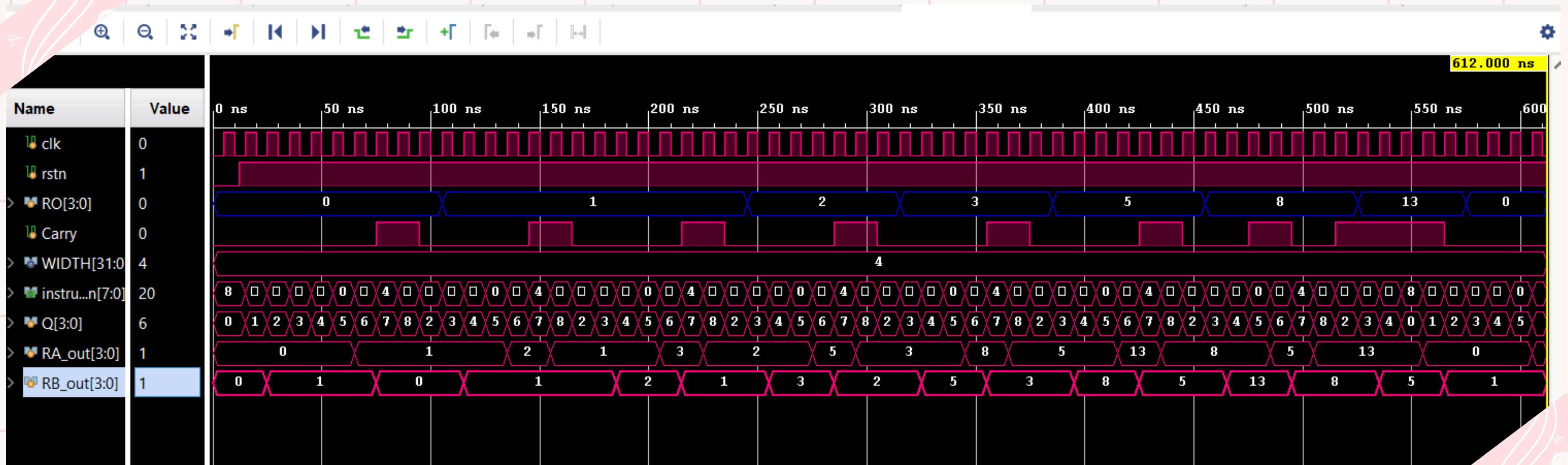
```
module topT #(
parameter WIDTH = 4,
parameter IMEM_DEPTH = 16,
parameter FILE_NAME =
"C:\\Users\\alfra\\Downloads\\machine_code.mem")(
input logic clk,
input logic rstn,
output logic [WIDTH-1:0] RO,
output logic Carry);
// Internal signals
logic [7:0] instruction;
logic J;
logic C;
logic D1;
logic D0;
logic Sreg;
logic S;
logic [2:0] imm;
//Datapath
datapath #(
.WIDTH(WIDTH),
.IMEM_DEPTH(IMEM_DEPTH),
.FILE_NAME(FILE_NAME))
DP (
.clk(clk),
.rstn(rstn),
.J(J),
.C(C),
.D1(D1),
.D0(D0),
.Sreg(Sreg),
.S(S),
.instruction(instruction),
.RO(RO),
.Carry(Carry),
.imm(imm));
// Control Unit
control_unit CU (
.instruction(instruction),
.J(J),
.C(C),
.D1(D1),
.D0(D0),
.Sreg(Sreg),
.S(S),
.imm(imm) );
endmodule
```

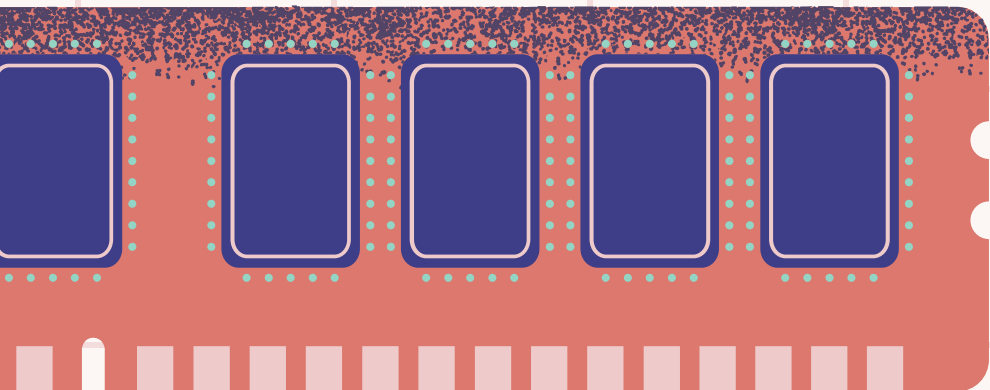
Datapath(datapath.sv)

```
odule datapath #(
parameter WIDTH = 4,
parameter IMEM_DEPTH = 16,
parameter FILE_NAME =
"C:\\Users\\alfra\\Downloads\\machine_code.
mem"
)(
input logic clk,
input logic rstn,
input logic J,
input logic C,
input logic D1,
input logic D0,
input logic Sreg,
input logic S,
// Immediate value
input logic [2:0] imm,
// Outputs
output logic [7:0] instruction,
output logic [WIDTH-1:0] RO ,
output logic Carry
);
// Program Counter output (Address)
logic [WIDTH-1:0] Q;
// Decoder outputs
logic en_RA;
logic en_RB;
logic en_RO;
// Register outputs
logic [WIDTH-1:0] RA_out;
logic [WIDTH-1:0] RB_out;
// MUX output
logic [WIDTH-1:0] MUX_out;
// ALU outputs
logic [WIDTH-1:0] ALU_out;
logic carry_wire;
PC #(
.WIDTH(WIDTH)
) PC (
.clk(clk),
.rstn(rstn),
.load(J | (C & Carry)),
.D({1'b0, imm}),
.Q(Q)
);
instruction_memory #(
.IMEM_DEPTH(IMEM_DEPTH),
.FILE_NAME(FILE_NAME)
) IM (
.address(Q),
.instruction(instruction)
);
```

```
Decoder DEC (
.D1(D1),
.D0(D0),
.y0(en_RA),
.y1(en_RB),
.y2(en_RO),
.y3()
);
file_registers #(
.WIDTH(WIDTH)
) RA_reg (
.clk(clk),
.rstn(rstn),
.en(en_RA),
.data_in(MUX_out),
.data_out(RA_out)
);
file_registers #(
.WIDTH(WIDTH)
) RB_reg (
.clk(clk),
.rstn(rstn),
.en(en_RB),
.data_in(MUX_out),
.data_out(RB_out)
);
file_registers #(
.WIDTH(WIDTH)
) RO_reg (
.clk(clk),
.rstn(rstn),
.en(en_RO),
.data_in(RA_out),
.data_out(RO)
);
Alu #(
.WIDTH(WIDTH)
) ALU (
.A(RA_out),
.B(RB_out),
.S(S),
.F(ALU_out),
.carry(carry_wire)
);
mux #(
.WIDTH(WIDTH)
) MUX1 (
.W0(ALU_out),
.W1({1'b0, imm}),
.Sel(Sreg),
.F(MUX_out)
);
file_registers #(
.WIDTH(1)
) dff1 (
.clk(clk),
.rstn(rstn),
.en(1'b1),
.data_in(carry_wire),
.data_out(Carry)
);
endmodule
```





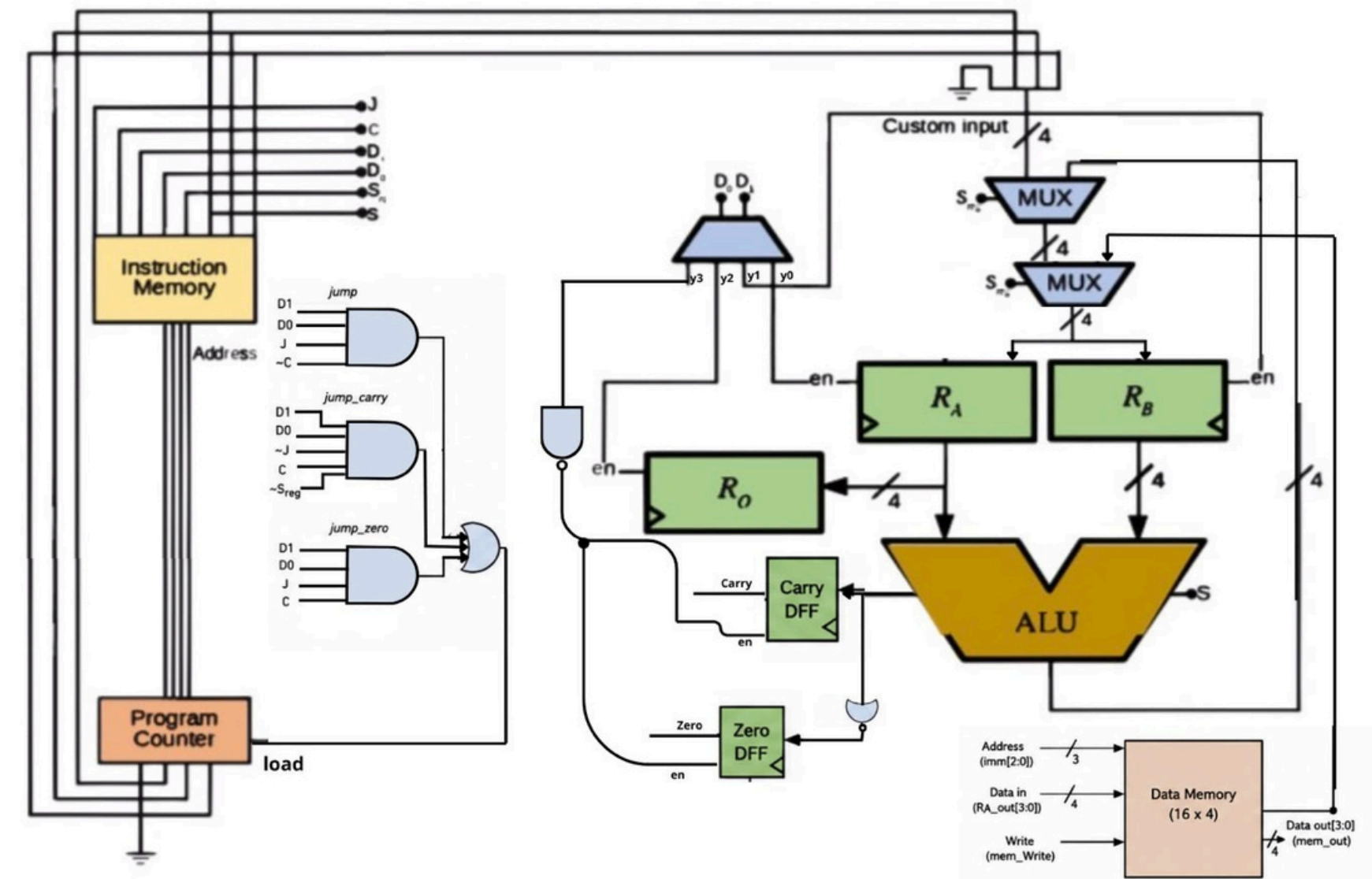


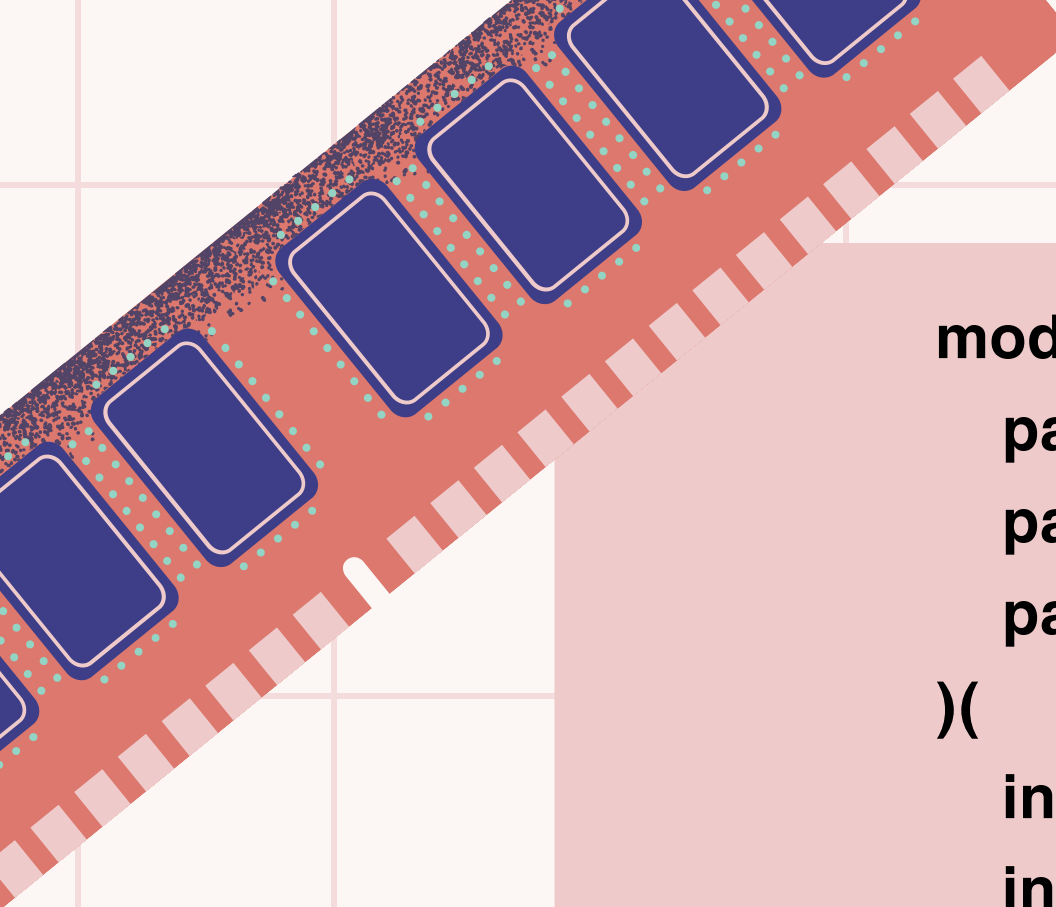
CONTROLLER DESIGN PHASE 2



COMPONENT (LOGIC CIRCUIT) USED PHASE-2

1. INSTRUCTION MEMORY
2. COUNTER (PC)
3. DECODER
4. MULTIPLEXER (MUX)
5. REGISTER R_0
6. REGISTER R_a
7. REGISTER R_b
8. ARITHMETIC LOGIC UNIT (ALU)
9. REGISTER (DFF)
10. CONTROLER
11. DATA MEMORY



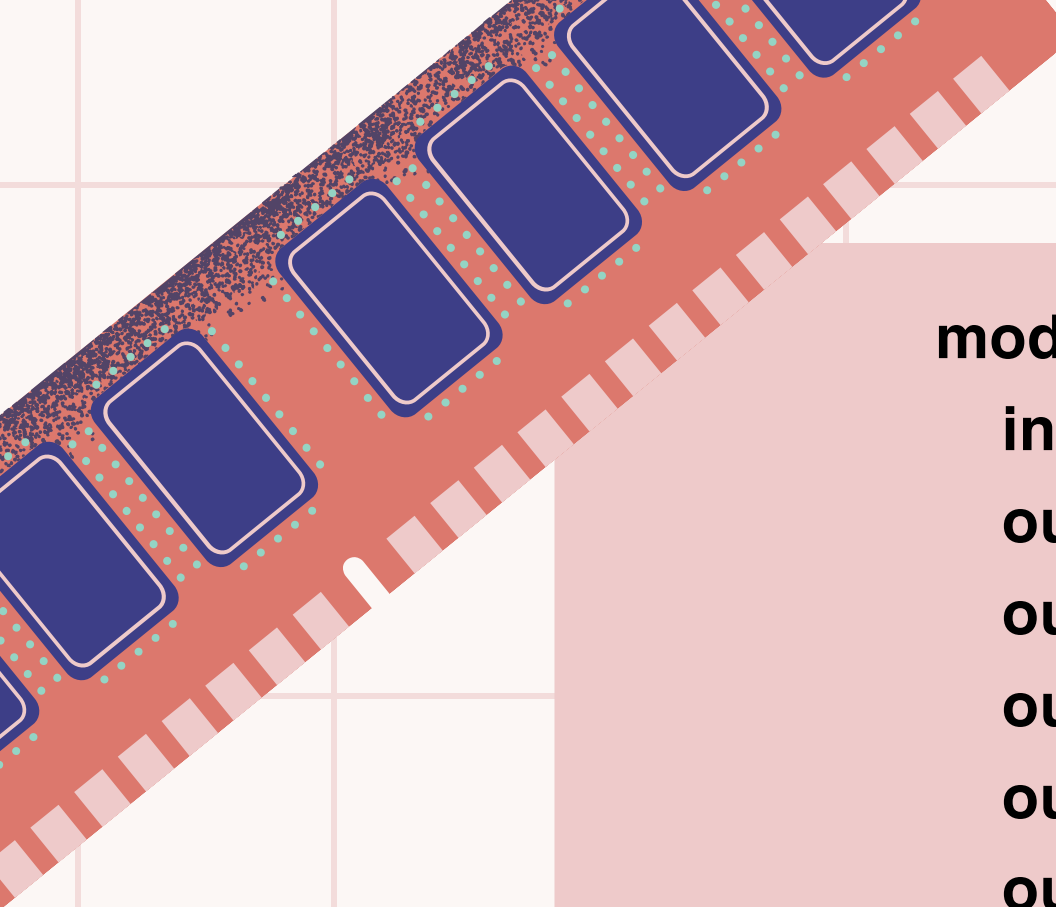


```
module data_memory #(
    parameter WIDTH = 4,
    parameter DEPTH = 8,
    parameter FILE_NAME = "C:\\Users\\alfra\\Downloads\\data.mem"
)
    input logic clk,
    input logic Write,
    input logic [$clog2(DEPTH)-1:0] address,
    input logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);

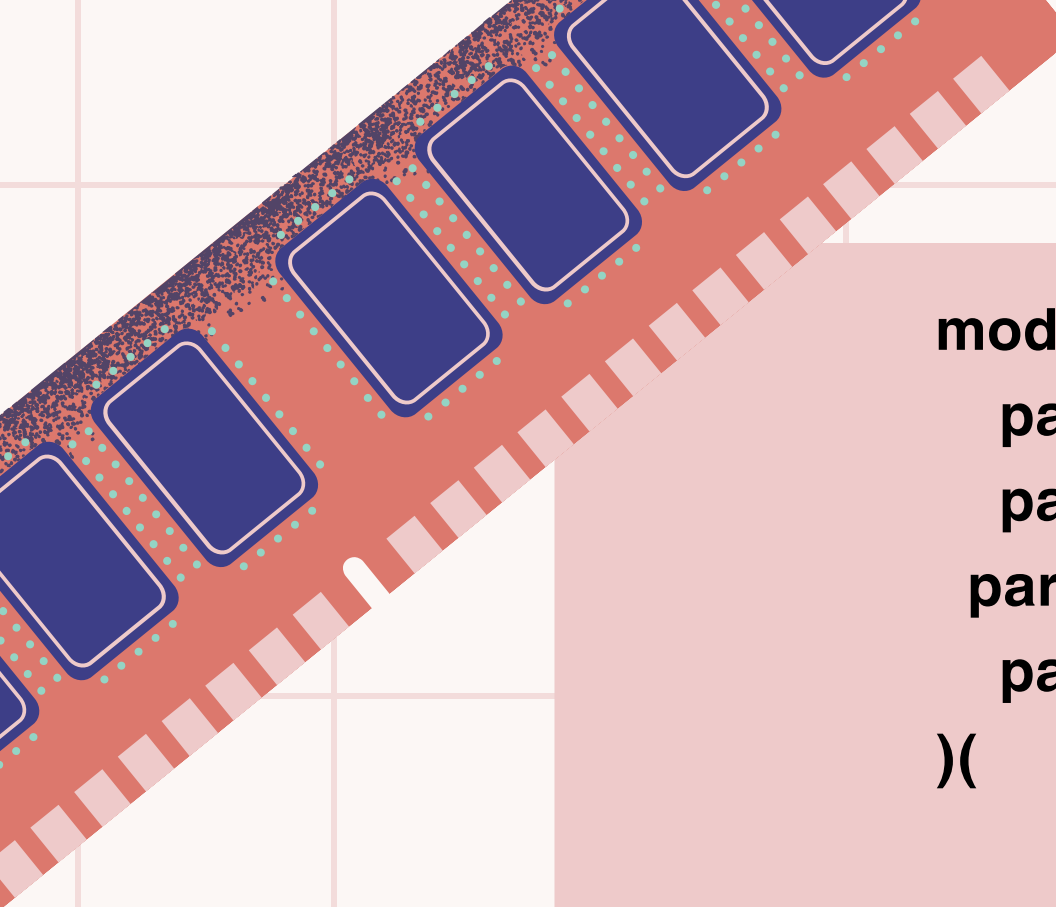
    logic [WIDTH-1:0] dmem [0:DEPTH-1];

    assign data_out = dmem[address];

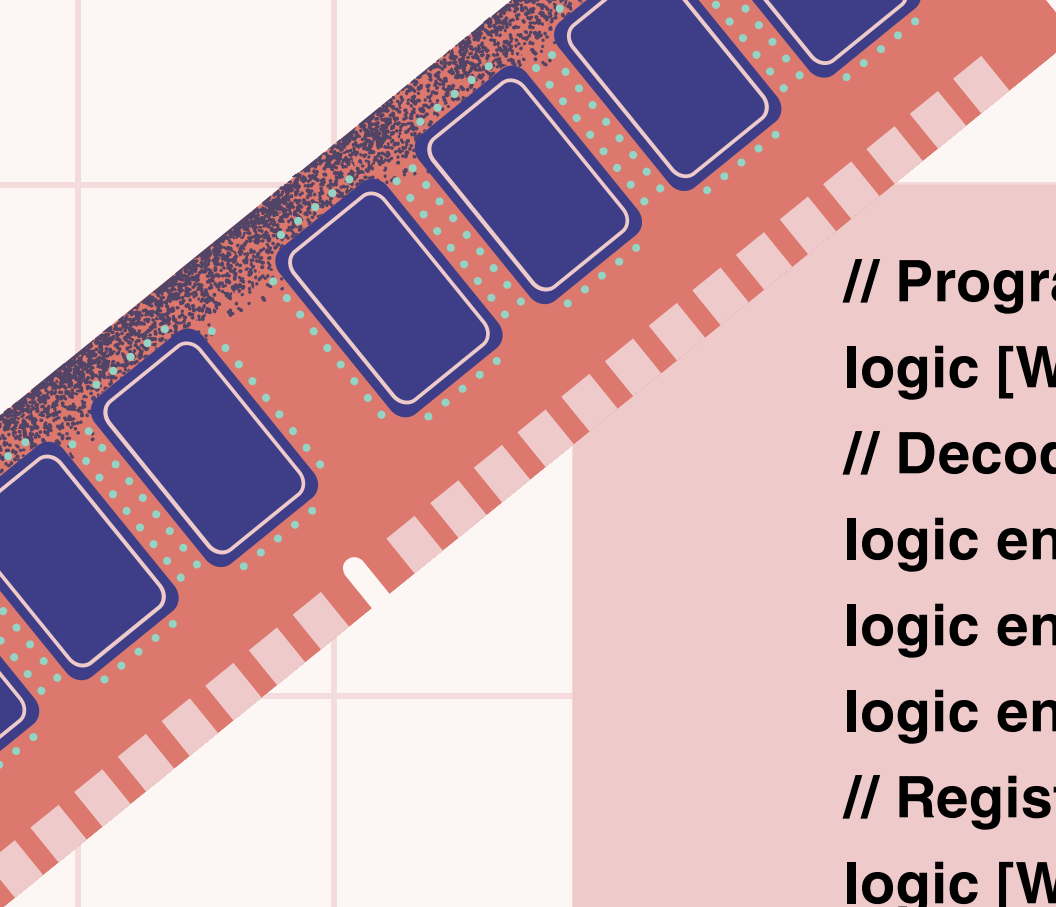
    always_ff @(posedge clk) begin
        if (Write)
            dmem[address] <= data_in;
    end
end
```



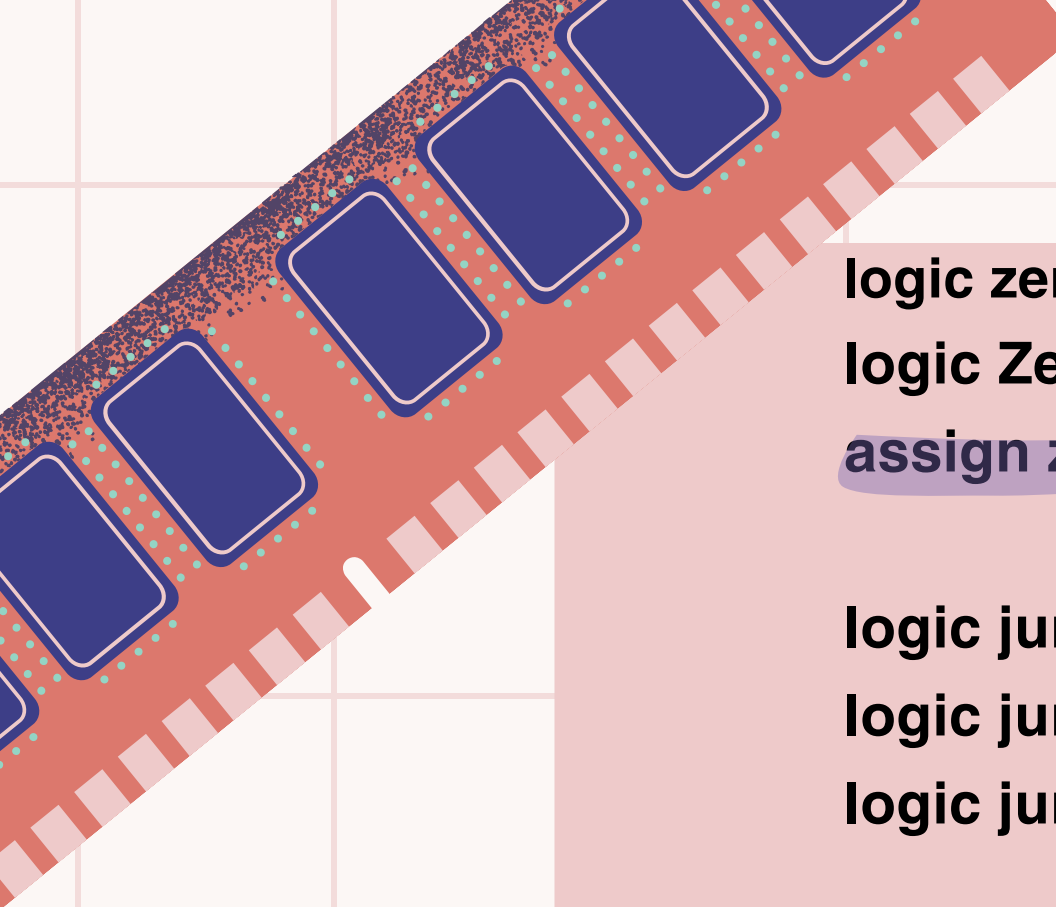
```
module control_unit (  
    input logic [7:0] instruction,  
    output logic J,  
    output logic C,  
    output logic D1,  
    output logic D0,  
    output logic Sreg,  
    output logic S,  
    output logic [2:0] imm  
);  
always_comb begin  
    J  = instruction[7];  
    C  = instruction[6];  
    D1 = instruction[5];  
    D0 = instruction[4];  
    Sreg = instruction[3];  
    S  = instruction[2];  
    imm = instruction[2:0];  
end  
endmodule
```



```
module datapath #(
    parameter WIDTH = 4,
    parameter IMEM_DEPTH = 16,
    parameter IMEM_FILE = "machine_code.mem",
    parameter DMEM_FILE = "data.mem"
)(
    input logic clk,
    input logic rstn,
    input logic J,
    input logic C,
    input logic D1,
    input logic D0,
    input logic Sreg,
    input logic S,
    // Immediate value
    input logic [2:0] imm,
    // Outputs
    output logic [7:0] instruction,
    output logic [WIDTH-1:0] RO ,
    output logic Carry
);
```

```
// Program Counter output (Address)
logic [WIDTH-1:0] Q;
// Decoder outputs
logic en_RA;
logic en_RB;
logic en_RO;
// Register outputs
logic [WIDTH-1:0] RA_out;
logic [WIDTH-1:0] RB_out;
// MUX output
logic [WIDTH-1:0] MUX_out;
// ALU outputs
logic [WIDTH-1:0] ALU_out;
logic carry_wire;
// data memory output
logic mem_Write;
logic RA_mem;
logic RB_mem;
assign RA_mem = (!J && C && !D1 && !D0 && Sreg); //RA=[imm]
assign RB_mem = (!J && C && !D1 && D0 && Sreg); //RB=[imm]
logic [3:0] mem_out;
assign mem_Write = (!J && C && D1 && D0 && Sreg);
```

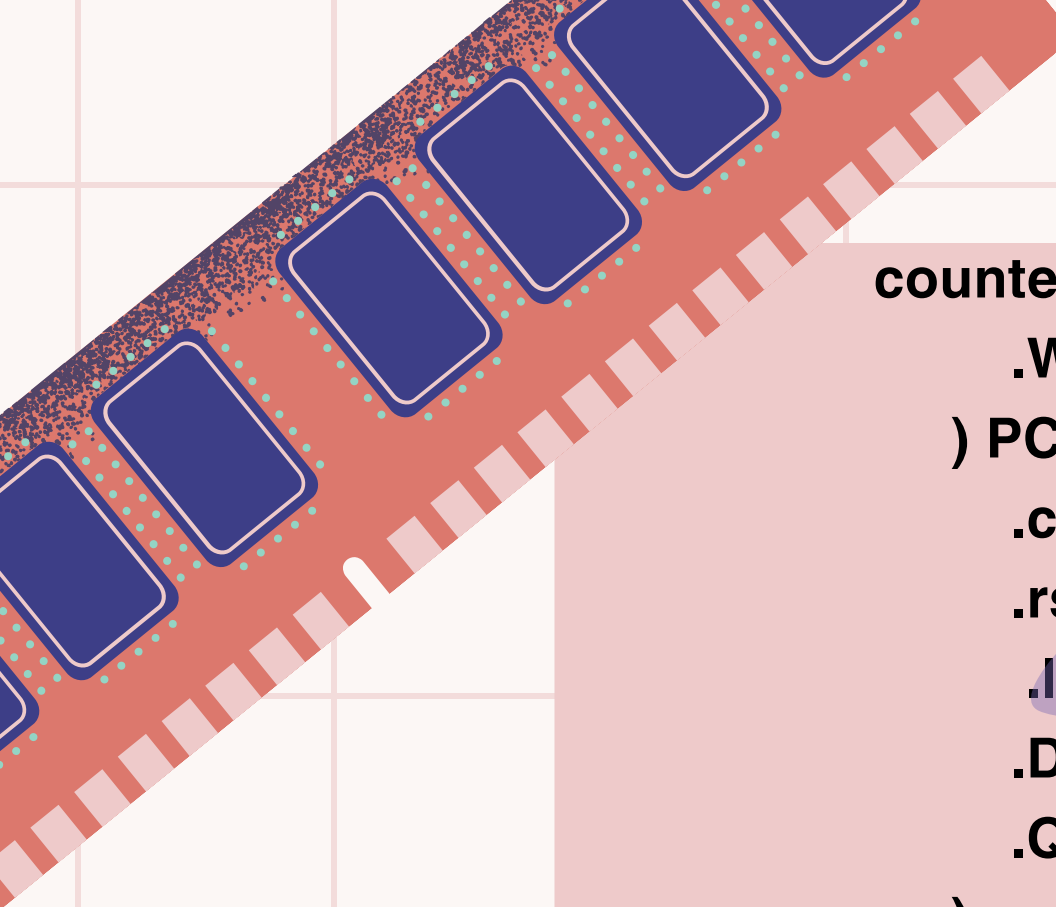


```
logic zero_wire;  
logic Zero;  
assign zero_wire = ~(IALU_out);
```

```
logic jump;  
logic jump_carry;  
logic jump_zero;
```

```
assign jump = D1 & D0 & J & ~C;  
assign jump_carry = D1 & D0 & ~J & C & ~Sreg;  
assign jump_zero = D1 & D0 & J & C;
```

```
data_memory #(  
    .WIDTH(WIDTH),  
    .DEPTH(8),  
    .FILE_NAME(DMEM_FILE)  
) DM (  
    .clk(clk),  
    .Write(mem_Write),  
    .address(imm),  
    .data_in(RA_out),  
    .data_out(mem_out));
```

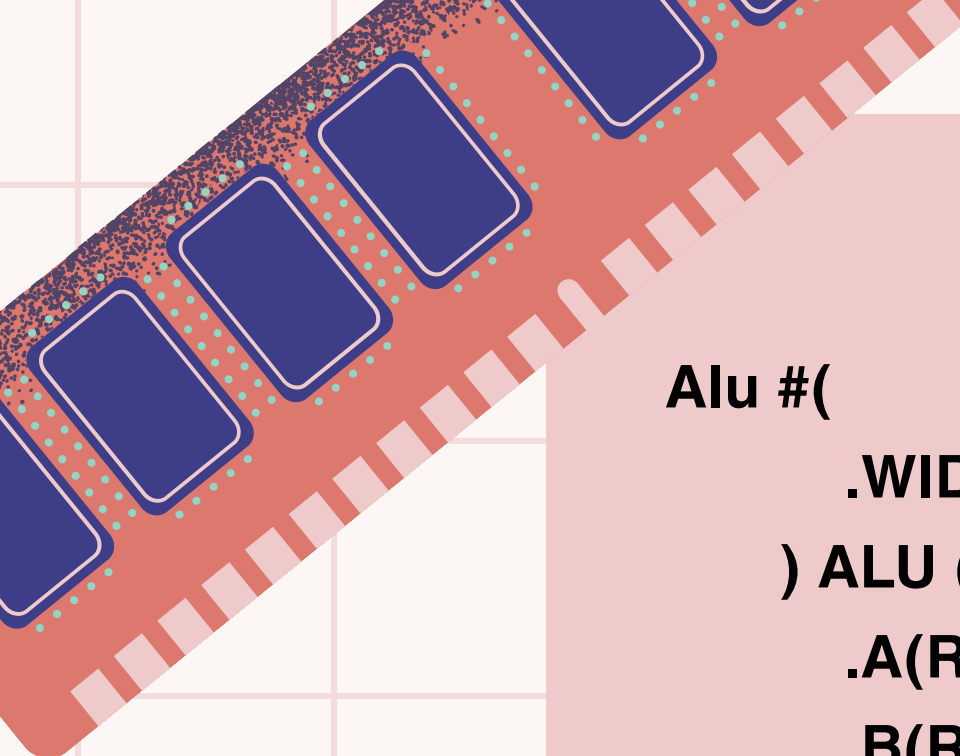



```
counter #(
    .WIDTH(WIDTH)
) PC (
    .clk(clk),
    .rstn(rstn),
    .load( jump_l(jump_carry & Carry)l(jump_zero & Zero) ),
    .D({1'b0, imm}),
    .Q(Q)
);
instruction_memory #(
    .IMEM_DEPTH(IMEM_DEPTH),
    .FILE_NAME(IMEM_FILE)
) IM (
    .address(Q),
    .instruction(instruction)
);
Decoder2 DEC (
    .D1(D1),
    .D0(D0),
    .y0(en_RA),
    .y1(en_RB),
    .y2(en_RO),
    .y3() );
```



```
registers #(
    .WIDTH(WIDTH)
) RA_reg (
    .clk(clk),
    .rstn(rstn),
    .en(en_RAI(RA_mem)),
    .data_in((RA_mem) ? mem_out : MUX_out),
    .data_out(RA_out)
);
registers #(
    .WIDTH(WIDTH)
) RB_reg (
    .clk(clk),
    .rstn(rstn),
    .en(en_RBI(RB_mem)),
    .data_in((RB_mem) ? mem_out : MUX_out),
    .data_out(RB_out)
);
```

```
registers #(
    .WIDTH(WIDTH)
) RO_reg (
    .clk(clk),
    .rstn(rstn),
    .en(en_RO && !C),
    .data_in(RA_out),
    .data_out(RO));
```



```
Alu #(
    .WIDTH(WIDTH)
) ALU (
    .A(RA_out),
    .B(RB_out),
    .S(S),
    .F(ALU_out),
    .carry(carry_wire)
);
mux #(
    .WIDTH(WIDTH)
) MUX (
    .W0(ALU_out),
    .W1({1'b0, imm}),
    .Sel(Sreg),
    .F(MUX_out)
);
```

```
registers #(
    .WIDTH(1)
) dff1 (
    .clk(clk),
    .rstn(rstn),
    .en(~(D1 & D0)),
    .data_in(carry_wire),
    .data_out(Carry)
);

registers #(
    .WIDTH(1)
) dff2 (
    .clk(clk),
    .rstn(rstn),
    .en(~(D1 & D0)),
    .data_in(zero_wire),
    .data_out(Zero)
);
endmodule
```

machine_code

Fibonacci Numbers (Phase 2 testing)

01001000 // RA=[0]

01111010 // [2]=RA

00100000 // RO=RA

01001001 // RA=[1]

01011010 // RB=[2]

00100000 // RO=RA

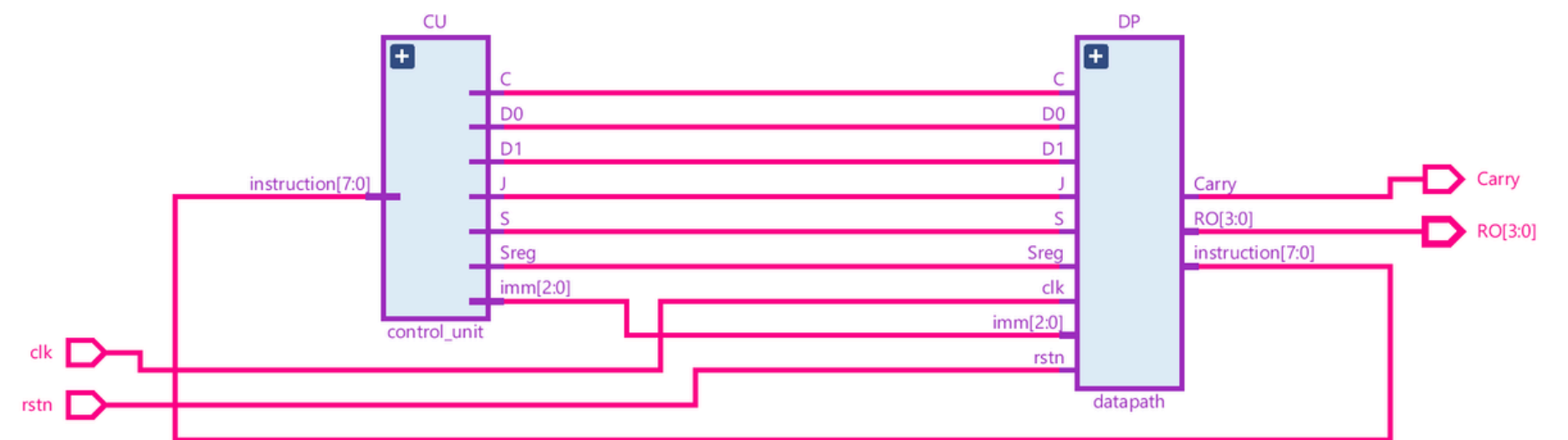
01111010 // [2]=RA

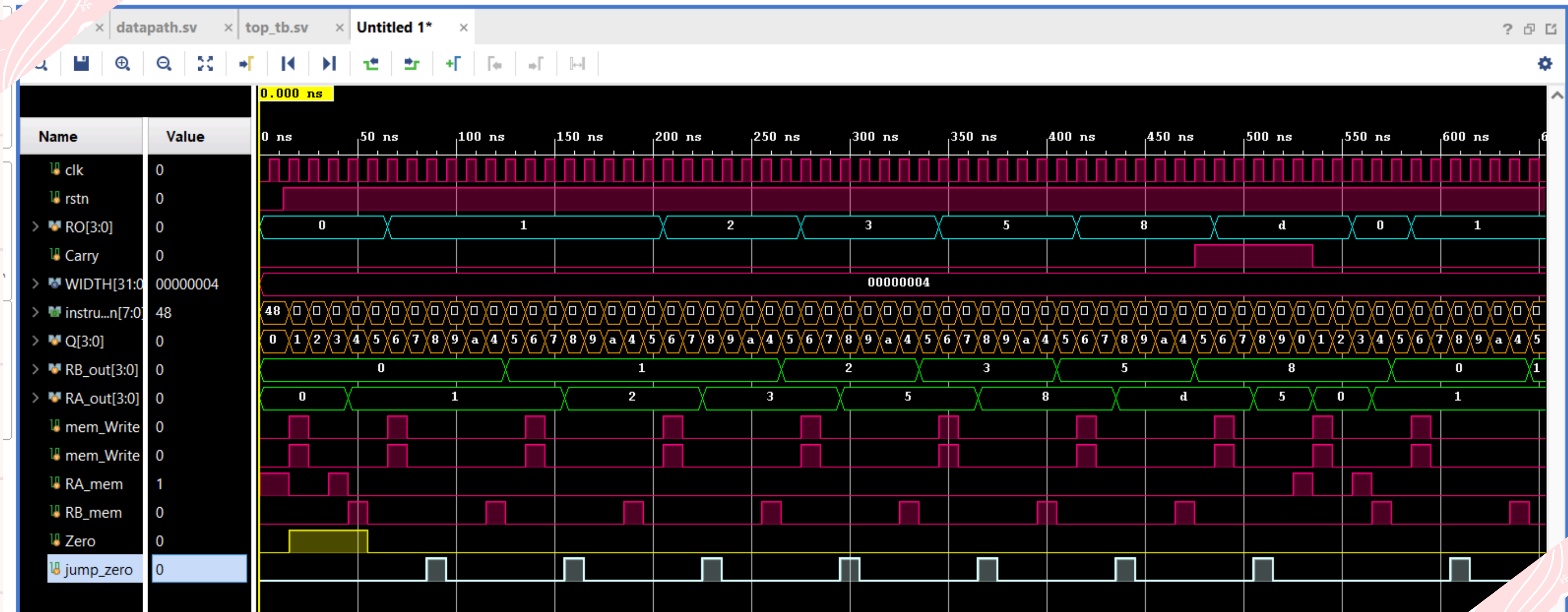
00000000 // RA=RA+RB

11111000 // JZ 0

01110000 // JC 0

10110100 // J 4







CONCLUSION

- **imm (3-bit) connected directly to 4-bit inputs – width mismatch** Zero-extended imm with {1'b0, imm}
- **Zero flag was combinational, not registered – JZ checked the wrong cycle's result** Added a DFF to register Zero, same as Carry
- **Store instruction accidentally triggered a Carry-jump** Added D1 & D0 & ~Sreg guard to jump_carry
- **Store instruction caused an unintended write to R0** Gated en_RO with && !C
- **Carry/Zero updated every cycle, even during jump instructions, overwriting correct values** Changed DFF enable to ~(D1 & D0) to freeze during jumps



THANK YOU

For Your Attention